

AtCoder Library (ACL)

Complete Usage Reference

Mathematics · Data Structures · Graphs · Strings

Ordered by increasing difficulty

Production version — C++14/C++17

Table of Contents

MATHEMATICS

1. modint — Modular Arithmetic [Beginner]
2. math — pow_mod, inv_mod, CRT [Easy]
3. math — floor_sum [Easy]
4. convolution — NTT Polynomial Multiply [Medium]

DATA STRUCTURES

5. fenwick_tree — Fenwick Tree / BIT [Beginner]
6. segtree — Segment Tree [Medium]
7. segtree — max_right / min_left [Medium]
8. lazysegtree — Lazy Segment Tree [Hard]
9. string — Suffix Array & LCP Array [Hard]
10. string — Z-Algorithm [Medium]

GRAPHS

11. dsu — Disjoint Set Union [Beginner]
12. scc — Strongly Connected Components [Medium]
13. twosat — 2-SAT Solver [Hard]
14. maxflow — Maximum Flow (Dinic) [Hard]
15. mincostflow — Min-Cost Max-Flow [Expert]

Chapter 1 — Mathematics

1. modint [Beginner] — #include

Handles modular arithmetic transparently. All operators (+, -, *, /) reduce mod m automatically. Use `modint998244353` or `modint1000000007` for the two most common contest primes; use `modint (dynamic)` when the modulus is given at runtime.

Type aliases

```
using mint = modint998244353; // static_modint<998244353>
using mint = modint1000000007; // static_modint<1000000007>
using mint = static_modint<1000000009>; // any other fixed prime
using mint = modint; // dynamic modulus (set at runtime)
```

Setting the modulus (dynamic only)

```
modint::set_mod(m); // call before any modint operation
// constraint: 1 <= m <= 2e9+1000
```

Constructors

```
mint a; // default -> 0
mint b = 10; // int/ll/ull/char/bool accepted; mod taken automatically
mint c(-1); // stores (mod - 1)
```

Methods

`int x.val()`

Returns the stored value in $[0, \text{mod})$. $O(1)$

`int mint::mod()`

Returns the current modulus. $O(1)$

`mint x.pow(ll n)`

Returns x^n . Constraint: $n \geq 0$. $O(\log n)$

`mint x.inv()`

Returns y with $x*y \equiv 1 \pmod m$. Constraint: $\gcd(x, \text{mod}) = 1$. $O(\log \text{mod})$

`mint mint::raw(int x)`

Constructs `modint(x)` WITHOUT taking mod — faster in tight loops when $x < \text{mod}$ is guaranteed. $O(1)$

Operators

```
// Unary
-mint; ++mint; --mint; mint++; mint--;

// Binary
mint + mint; mint - mint; mint * mint; mint / mint;
mint += mint; mint -= mint; mint *= mint; mint /= mint;
mint == mint; mint != mint;

// Mixed: int/ll on either side is promoted automatically
1 + mint_x; mint_y * 5;
```

Multiple independent mods

```
using mint0 = dynamic_modint<0>;
using mint1 = dynamic_modint<1>;
// Each tag is independent; modint is dynamic_modint<-1>
```

Full working example

```
#include <atcoder/modint>
#include <cstdio>
using namespace std;
using namespace atcoder;
using mint = static_modint<11>;

int main() {
    mint a = 10, b(3);
    assert(a == 21); // 21 % 11 == 10
    assert(a == -1); // -1 mod 11 == 10
    assert(-b == 8);
    assert(a + b == 2); // (10+3) mod 11
    assert(a - b == 7);
    assert(a * b == 8); // (10*3) mod 11
    assert(b.inv() == 4); // 3*4 mod 11 == 1
    assert(a / b == 7); // 10 * inv(3) mod 11
    assert(mint(2).pow(4) == 5); // 16 mod 11
    printf("%d\n", a.val()); // 10

    // Fast accumulation when i < mod is guaranteed
    mint sum = 0;
    for (int i = 0; i < 100000; i++)
        sum += mint::raw(i);
}
```

Typical contest usage

```
#include <atcoder/modint>
#include <iostream>
using namespace std;
using namespace atcoder;
using mint = modint998244353;

int main() {
    int n; cin >> n;
    mint ans = 0;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        ans += x;
    }
```

```

}

cout << ans.val() << "\n";

}

```

Warning: Division a/b requires $\gcd(b.val(), \text{mod}) == 1$. Do not use division with non-prime moduli.

2. math — pow_mod / inv_mod / CRT [Easy] — #include

Number-theoretic functions on plain long long. No modint needed.

pow_mod

```

ll pow_mod(ll x, ll n, int m)
// Returns  $x^n \bmod m$ . Constraints:  $n \geq 0$ ,  $m \geq 1$ .  $O(\log n)$ 

pow_mod(2, 10, 1000); //  $1024 \bmod 1000 = 24$ 
pow_mod(3, 0, 7); // 1

```

inv_mod

```

ll inv_mod(ll x, ll m)
// Returns  $y$  in  $[0, m)$  with  $x*y \equiv 1 \pmod{m}$ .
// Constraint:  $\gcd(x, m) == 1$ ,  $m \geq 1$ .  $O(\log m)$ 

inv_mod(3, 7); // 5 ( $3*5 = 15 \equiv 1 \pmod{7}$ )

```

crt — Chinese Remainder Theorem

```

pair<ll, ll> crt(vector<ll> r, vector<ll> m)
// Solves  $x \equiv r[i] \pmod{m[i]}$  for all  $i$ .
// Returns  $(y, z)$ : unique solution  $y$  in  $[0, z)$ ,  $z = \text{lcm}(m[i])$ .
// Returns  $(0, 0)$  if no solution.  $O(n * \log(\text{lcm}(m[i])))$ 

auto [y, z] = crt({2, 3}, {3, 5});
//  $x \equiv 2 \pmod{3}$  and  $x \equiv 3 \pmod{5}$ 
//  $y = 8$ ,  $z = 15 \Rightarrow x \equiv 8 \pmod{15}$ 

```

Note: crt handles pairwise non-coprime moduli automatically. The lcm must fit in ll.

Example

```

#include <atcoder/math>
#include <cstdio>
using namespace atcoder;

int main() {
    printf("%lld\n", pow_mod(2LL, 60LL, 1000000007));
    printf("%lld\n", inv_mod(3LL, 1000000007));

    auto [y, z] = crt({1, 2}, {4, 6});
    if (z == 0) puts("No solution");
    else printf("x == %lld (mod %lld)\n", y, z);
}

```

```
}
```

3. math — floor_sum [Easy] — #include

Computes $\sum_{i=0}^{n-1} \text{floor}((a*i + b) / m)$ in $O(\log m)$. Useful for lattice-point counting under a line and floor-sum DP transitions.

```
ll floor_sum(ll n, ll m, ll a, ll b)
// Constraints: 0 <= n < 2^32, 1 <= m < 2^32. 0 < log m
// Result returned mod 2^64 on overflow.
```

AC example (practice2/c)

```
#include <atcoder/math>
#include <cstdio>
using namespace atcoder;

int main() {
    int t; scanf("%d", &t);
    while (t--) {
        long long n, m, a, b;
        scanf("%lld %lld %lld %lld", &n, &m, &a, &b);
        printf("%lld\n", floor_sum(n, m, a, b));
    }
}
```

4. convolution — NTT Polynomial Multiply [Medium] — #include

Computes $c[i] = \sum_{j=0}^i a[j] * b[i-j]$ using Number-Theoretic Transform. Returns array of length $|a|+|b|-1$.

```
// (1) Integer arrays, result mod m (default m = 998244353)
vector<T> convolution<int m = 998244353>(vector<T> a, vector<T> b)
// T: int, uint, ll, ull.

// (2) modint arrays
vector<static_modint<m>> convolution<int m>(
    vector<static_modint<m>> a, vector<static_modint<m>> b)

// (3) Result fits in ll, no modular reduction
vector<ll> convolution_ll(vector<ll> a, vector<ll> b)
// |a|+|b|-1 <= 2^24. Uses 3-prime NTT + CRT.

// Complexity: O(n log n + log mod), n = |a| + |b|
```

Note: m must be prime with $2^c \mid (m-1)$ and $|a|+|b|-1 \leq 2^c$. $998244353 = 119 \cdot 2^{23} + 1$ supports arrays up to length 2^{23} .

Example 1 — plain integer arrays

```
#include <atcoder/convolution>
#include <cstdio>
```

```

using namespace atcoder;

int main() {
    int n, m; scanf("%d %d", &n, &m);
    vector<long long> a(n), b(m);
    for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
    for (int i = 0; i < m; i++) scanf("%lld", &b[i]);
    auto c = convolution(a, b);
    for (int i = 0; i < n+m-1; i++) printf("%lld ", c[i]);
}

```

Example 2 — with modint

```

#include <atcoder/convolution>
#include <atcoder/modint>
using namespace atcoder;
using mint = modint998244353;

int main() {
    int n, m; scanf("%d %d", &n, &m);
    vector<mint> a(n), b(m);
    for (int i = 0; i < n; i++) { int x; scanf("%d",&x); a[i]=x; }
    for (int i = 0; i < m; i++) { int x; scanf("%d",&x); b[i]=x; }
    auto c = convolution(a, b);
    for (int i = 0; i < n+m-1; i++) printf("%d ", c[i].val());
}

```

Warning: Returns empty array if either input is empty.

Chapter 2 — Data Structures

5. fenwick_tree — BIT / Fenwick Tree [Beginner] — #include

Point update and prefix-sum query in $O(\log n)$. Simpler and faster constant than segtree when only sums are needed.

```
fenwick_tree<T> fw(int n)
// Creates array a[0..n-1] all zero.
// T: int, uint, ll, ull, or modint.  $0 \leq n \leq 1e8$ .  $O(n)$ .
```

Methods

void fw.add(int p, T x)

$a[p] += x$. Constraint: $0 \leq p < n$. $O(\log n)$

T fw.sum(int l, int r)

Returns $a[l] + \dots + a[r-1]$. Constraint: $0 \leq l \leq r \leq n$. $O(\log n)$

AC example (practice2/b)

```
#include <atcoder/fenwicktree>
#include <cstdio>
using namespace atcoder;

int main() {
    int n, q; scanf("%d %d", &n, &q);
    fenwick_tree<long long> fw(n);
    for (int i = 0; i < n; i++) {
        int a; scanf("%d", &a); fw.add(i, a);
    }
    for (int i = 0; i < q; i++) {
        int t; scanf("%d", &t);
        if (t == 0) {
            int p, x; scanf("%d %d", &p, &x); fw.add(p, x);
        } else {
            int l, r; scanf("%d %d", &l, &r);
            printf("%lld\n", fw.sum(l, r));
        }
    }
}
```

Pattern — count inversions

```
fenwick_tree<int> fw(MAXV);
long long inv = 0;
for (int i = n-1; i >= 0; i--) {
    inv += fw.sum(0, a[i]); // elements already seen that are smaller
    fw.add(a[i], 1);
}
```

Pattern — binary search on BIT (k-th element)

```
int find_kth(fenwick_tree<int>& fw, int n, int k) {
    int pos = 0;
    for (int pw = 1 << 20; pw; pw >>= 1)
        if (pos + pw <= n && fw.sum(0, pos + pw) < k) pos += pw;
    return pos;
}
```

6. segtree — Segment Tree [Medium] — #include

Generic segment tree for any monoid (S, op, e). Point-set and range-product in $O(\log n)$. You define the type, binary operation, and identity — the library does the rest.

What you must define

```
// 1. Value type S
// 2. Associative binary operation
S op(S a, S b) { ... }
// 3. Identity element: op(e(), x) == op(x, e()) == x for all x
S e() { return ...; }

segtree<S, op, e> seg(int n); // all elements = e()
segtree<S, op, e> seg(vector<S> v); // initialise from vector
```

Methods

void seg.set(int p, S x)

$a[p] = x$. $O(\log n)$

S seg.get(int p)

Returns $a[p]$. $O(1)$

S seg.prod(int l, int r)

Returns $op(a[l], \dots, a[r-1])$. Returns $e()$ if $l=r$. $O(\log n)$

S seg.all_prod()

Returns $op(a[0], \dots, a[n-1])$. $O(1)$

Common monoid recipes

```
// Range Maximum
int op(int a, int b) { return max(a, b); } int e() { return INT_MIN; }

// Range Minimum
int op(int a, int b) { return min(a, b); } int e() { return INT_MAX; }

// Range Sum
long long op(long long a, long long b) { return a + b; } long long e() { return 0; }

// Range GCD
long long op(long long a, long long b) { return __gcd(a, b); } long long e() { return 0; }
```

```
// Range Bitwise AND
int op(int a, int b) { return a & b; } int e() { return ~0; }
```

AC example (practice2/i) — RMQ + point update

```
#include <atcoder/segtree>
#include <cstdio>
using namespace atcoder;

int op(int a, int b) { return max(a, b); }
int e() { return -1; }

int target;
bool f(int v) { return v < target; }

int main() {
    int n, q; scanf("%d %d", &n, &q);
    vector<int> a(n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    segtree<int, op, e> seg(a);

    for (int i = 0; i < q; i++) {
        int t; scanf("%d", &t);
        if (t == 1) { int x, v; scanf("%d %d",&x,&v); seg.set(x-1, v); }
        else if (t == 2) { int l, r; scanf("%d %d",&l,&r); printf("%d\n", seg.prod(l-1,r)); }
        else { int p; scanf("%d %d",&p,&target); printf("%d\n", seg.max_right<f>(p-1)+1); }
    }
}
```

7. segtree — max_right / min_left [Medium] — Binary Search on Tree

Find the longest prefix/suffix satisfying a monotone predicate in $O(\log n)$. Replaces the classic binary search + range query pattern that costs $O(\log^2 n)$.

```
// max_right: returns largest r such that
// f(op(a[l], ..., a[r-1])) == true
// AND (r==n OR f(op(a[l], ..., a[r])) == false)
int seg.max_right<f>(int l) // f is a global bool function
int seg.max_right<F>(int l, F f) // F is a lambda or functor

// Invariants: f must be monotone; f(e()) must be true; 0 <= l <= n.
// Complexity: O(log n)

// min_left: symmetric, returns smallest l
int seg.min_left<f>(int r)
int seg.min_left<F>(int r, F f)
```

Pattern — first position where prefix sum $\geq k$

```
long long K;
```

```
bool f(long long x) { return x < K; }

int pos = seg.max_right<f>(0);

// pos is the first index where prefix sum from 0 first reaches K
```

Lambda version (C++17)

```
int target = 5;

int pos = seg.max_right(l, [&](int v) { return v < target; });
```

8. lazysegtree — Lazy Segment Tree [Hard] — #include

Extends segtree with range updates (lazy propagation). You define a monoid (S, op, e) AND a set F of maps $S \rightarrow S$ that: (1) contains identity, (2) is closed under composition, (3) distributes over op . Both range-apply and range-product run in $O(\log n)$.

What you must define

```
struct S { ... }; // value type

S op(S a, S b) { ... } // associative, must distribute with mapping

S e() { ... } // identity of op

struct F { ... }; // map type

S mapping(F f, S x) { return f(x); } // apply map f to value x

F composition(F f, F g) { return f ◦ g; } // f applied after g

F id() { return ...; } // identity map: id()(x) == x

lazy_segtree<S, op, e, F, mapping, composition, id> seg(n);

lazy_segtree<S, op, e, F, mapping, composition, id> seg(vector<S> v);
```

Additional method: apply

```
void seg.apply(int p, F f) // a[p] = f(a[p])  $O(\log n)$ 

void seg.apply(int l, int r, F f) // a[i] = f(a[i]) for  $l \leq i < r$   $O(\log n)$ 
```

Example 1 — Range affine transform ($a \cdot x + b$), range sum

```
using mint = modint998244353;

struct S { mint val; int size; };

struct F { mint a, b; }; // map:  $x \rightarrow a \cdot x + b$ 

S op(S l, S r) { return {l.val + r.val, l.size + r.size}; }

S e() { return {0, 0}; }

S mapping(F f, S r) { return {r.val * f.a + r.size * f.b, r.size}; }

F composition(F f, F g) { return {g.a * f.a, g.b * f.a + f.b}; }

F id() { return {1, 0}; } //  $x \rightarrow 1 \cdot x + 0$ 

// Build

vector<S> init(n);

for (int i = 0; i < n; i++) init[i] = {a[i], 1};

lazy_segtree<S, op, e, F, mapping, composition, id> seg(init);
```

```
// Range apply: a[i] = c*a[i]+d for i in [l, r)
seg.apply(l, r, F{c, d});

// Range sum
seg.prod(l, r).val.val();
```

Example 2 — Range flip 0/1 array, query inversions

```
struct S { long long zero, one, inversion; };
using F = bool; // flip flag

S op(S l, S r) { return {l.zero+r.zero, l.one+r.one,
l.inversion+r.inversion + l.one*r.zero}; }
S e() { return {0,0,0}; }
S mapping(F fl, S r) {
if (!fl) return r;
return {r.one, r.zero, r.one*r.zero - r.inversion};
}
F composition(F f, F g) { return f ^ g; } // two flips cancel
F id() { return false; }

// Build from 01-array
vector<S> init(n);
for (int i = 0; i < n; i++)
init[i] = (a[i]==0) ? S{1,0,0} : S{0,1,0};
lazy_segtree<S,op,e,F,mapping,composition,id> seg(init);

seg.apply(l, r, true); // flip [l, r)
seg.prod(l, r).inversion; // count inversions in [l, r)
```

Quick-reference — common lazy patterns

```
// (A) Range-add, range-sum
// S = ll (value + size packed), F = ll (add amount)
// mapping(f, x) = x.val + f * x.size
// composition(f, g) = f + g, id = 0

// (B) Range-assign, range-min
// S = ll, F = optional<ll>
// mapping(f, x) = f.has_value() ? *f : x
// composition(f, g) = f.has_value() ? f : g, id = nullopt

// (C) Range-multiply, range-sum (modint)
// S = {sum, size}, F = mint multiplier
// mapping(f, x) = {x.sum * f, x.size}
// composition = multiply, id = 1
```

9. string — Suffix Array & LCP Array [Hard] — #include

`suffix_array(s)` returns the sorted order of all suffixes of `s`. `lcp_array(s, sa)` returns LCP lengths of consecutive sorted suffixes. Together they enable $O(n)$ / $O(n \log n)$ solutions to many string problems.

```
vector<int> suffix_array(string s) // O(n) SA-IS
vector<int> suffix_array<T>(vector<T> s) // O(n log n)
vector<int> suffix_array(vector<int> s, int upper) // O(n + upper)

vector<int> lcp_array(string s, vector<int> sa) // O(n)
// lcp[i] = length of LCP of s[sa[i]..n) and s[sa[i+1]..n)
```

AC example — count distinct substrings

```
#include <atcoder/string>
#include <cstdio>
using namespace atcoder;

int main() {
    char buf[500001]; scanf("%s", buf);
    string s = buf;
    int n = s.size();
    auto sa = suffix_array(s);
    auto lcp = lcp_array(s, sa);
    long long ans = 1LL * n * (n+1) / 2;
    for (int x : lcp) ans -= x; // subtract duplicate substrings
    printf("%lld\n", ans);
}
```

Pattern — binary search for a pattern in the suffix array

```
auto sa = suffix_array(t);
// Find all occurrences of pattern p in text t
auto lo = lower_bound(sa.begin(), sa.end(), 0,
    [&](int a, int) { return t.substr(a, p.size()) < p; });
auto hi = upper_bound(sa.begin(), sa.end(), 0,
    [&](int, int a) { return p < t.substr(a, p.size()); });
// occurrences are sa[lo..hi)
```

10. string — Z-Algorithm [Medium] — #include

`z[i]` = length of longest prefix of `s` that matches `s[i..n)`. `z[0] = n`. Use for linear-time pattern matching, period detection, and prefix queries.

```
vector<int> z_algorithm(string s) // O(n)
vector<int> z_algorithm<T>(vector<T> s) // O(n), T: int/uint/ll/ull
```

Pattern — find all occurrences of `p` in `t`

```
#include <atcoder/string>
using namespace atcoder;
```

```
vector<int> find_all(const string& p, const string& t) {
    string s = p + "#" + t; // separator not in p or t
    auto z = z_algorithm(s);
    int m = p.size();
    vector<int> res;
    for (int i = m+1; i < (int)s.size(); i++)
        if (z[i] >= m) res.push_back(i - m - 1);
    return res; // 0-indexed positions in t
}
```

Pattern — smallest period

```
auto z = z_algorithm(s);
int n = s.size();
for (int p = 1; p < n; p++)
    if (n % p == 0 && z[p] == n - p) {
        // s has smallest period p
        break;
    }
```

Chapter 3 — Graphs

11. dsu — Disjoint Set Union [Beginner] — #include

Union-Find with path compression and union by size. Edge addition and connectivity queries in $O(\alpha(n))$ amortised. `groups()` enumerates all connected components.

```
dsu d(int n)
// Undirected graph, n vertices, 0 edges.  $O(n)$ .
```

Methods

int d.merge(int a, int b)

Adds edge (a,b). Returns representative of merged component. $O(\alpha(n))$

bool d.same(int a, int b)

Returns true iff a and b are in the same component. $O(\alpha(n))$

int d.leader(int a)

Returns representative (root) of a's component. $O(\alpha(n))$

int d.size(int a)

Returns size of a's component. $O(\alpha(n))$

vector<vector<int>> d.groups()

Returns all components as lists of vertices. Order undefined. $O(n)$

AC example (practice2/a)

```
#include <atcoder/dsu>
#include <cstdio>
using namespace atcoder;

int main() {
    int n, q; scanf("%d %d", &n, &q);
    dsu d(n);
    for (int i = 0; i < q; i++) {
        int t, u, v; scanf("%d %d %d", &t, &u, &v);
        if (t == 0) d.merge(u, v);
        else printf("%d\n", d.same(u, v) ? 1 : 0);
    }
}
```

Pattern — Kruskal MST

```
sort(edges.begin(), edges.end()); // sort by weight
dsu d(n);
long long mst = 0;
for (auto [w, u, v] : edges)
    if (!d.same(u, v)) { d.merge(u, v); mst += w; }
```

Enumerate all components

```

for (auto& comp : d.groups()) {
    // comp is a vector<int> of vertices in this component
}

```

12. scc — Strongly Connected Components [Medium] — #include

Computes SCCs of a directed graph in $O(V + E)$. Returns components in topological order: if a path goes from component A to B ($A \neq B$), then A appears before B.

```

scc_graph g(int n) // n vertices, 0 edges. O(n)
void g.add_edge(int from, int to) // directed edge. O(1) amortised

vector<vector<int>> g.scc() // O(n + m)
// Returns list of SCCs in topological order, each as a list of vertices.

```

AC example (practice2/g)

```

#include <atcoder/scc>
#include <cstdio>
using namespace atcoder;

int main() {
    int n, m; scanf("%d %d", &n, &m);
    scc_graph g(n);
    for (int i = 0; i < m; i++) {
        int u, v; scanf("%d %d", &u, &v); g.add_edge(u, v);
    }
    auto comp = g.scc();
    printf("%d\n", (int)comp.size());
    for (auto& v : comp) {
        printf("%d", (int)v.size());
        for (int x : v) printf(" %d", x);
        printf("\n");
    }
}

```

Pattern — condensation DAG DP

```

auto comps = g.scc();
int K = comps.size();
vector<int> cid(n);
for (int c = 0; c < K; c++)
    for (int v : comps[c]) cid[v] = c;

// comps are already in topological order (index 0 = source side)
vector<long long> dp(K, 0);
for (int c = 0; c < K; c++) {
    dp[c] = ...; // base case for this component
}

```

```
// propagate to neighbors using cid[]
}
```

13. twosat — 2-SAT Solver [Hard] — #include

Solves 2-SAT: given N boolean variables and clauses $(x_i == f) \text{ OR } (x_j == g)$, decides satisfiability in $O(N + M)$ using an implication graph + SCC.

```
two_sat ts(int n) // n variables, 0 clauses. O(n)
void ts.add_clause(int i, bool f, int j, bool g) // adds  $(x_i == f) || (x_j == g)$ . O(1)
bool ts.satisfiable() // O(n + m). Call before answer().
vector<bool> ts.answer() // valid assignment. O(n)
```

Clause encoding table

```
// Meaning add_clause call
// x_i must be true add_clause(i, true, i, true)
// x_i must be false add_clause(i, false, i, false)
// x_i implies x_j add_clause(i, false, j, true)
// x_i and x_j cannot both be true add_clause(i, false, j, false)
// x_i and x_j cannot both be false add_clause(i, true, j, true)
// at least one of x_i, x_j is true add_clause(i, true, j, true)
```

AC example (practice2/h) — flag placement

```
#include <atcoder/twosat>
#include <iostream>
using namespace atcoder;

int main() {
    int n, d; cin >> n >> d;
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++) cin >> x[i] >> y[i];

    // ts[i] = true -> flag i at x[i]
    // ts[i] = false -> flag i at y[i]
    two_sat ts(n);
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++) {
            if (abs(x[i]-x[j]) < d) ts.add_clause(i, false, j, false);
            if (abs(x[i]-y[j]) < d) ts.add_clause(i, false, j, true);
            if (abs(y[i]-x[j]) < d) ts.add_clause(i, true, j, false);
            if (abs(y[i]-y[j]) < d) ts.add_clause(i, true, j, true);
        }
    if (!ts.satisfiable()) { cout << "No\n"; return 0; }
    cout << "Yes\n";
    auto ans = ts.answer();
}
```

```

for (int i = 0; i < n; i++)
    cout << (ans[i] ? x[i] : y[i]) << "\n";
}

```

14. maxflow — Maximum Flow (Dinic) [Hard] — #include

Dinic's algorithm for max s-t flow. Supports capacity inspection, edge flow queries, and min-cut extraction.

```

mf_graph<Cap> g(int n) // Cap: int or ll. O(n)

int g.add_edge(int from, int to, Cap cap) // returns edge index. O(1)

Cap g.flow(int s, int t) // max flow. O(n^2 m)

Cap g.flow(int s, int t, Cap limit) // stops at limit

vector<bool> g.min_cut(int s) // s-side of min cut. O(n+m)

mf_graph<Cap>::edge g.get_edge(int i) // {from,to,cap,flow}. O(1)

vector<mf_graph<Cap>::edge> g.edges() // all edges. O(m)

```

Note: Complexity: $O(\min(n^{2/3}m, m^{3/2}))$ for unit capacities; $O(n^2m)$ in general.

AC example (practice2/d) — bipartite matching on grid

```

#include <atcoder/maxflow>
#include <iostream>
using namespace atcoder;

int main() {
    int n, m; cin >> n >> m;
    vector<string> grid(n);
    for (auto& row : grid) cin >> row;

    mf_graph<int> g(n*m + 2);
    int s = n*m, t = n*m+1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == '#') continue;
            int v = i*m+j;
            if ((i+j)%2 == 0) g.add_edge(s, v, 1);
            else g.add_edge(v, t, 1);
        }

    int dx[] = {1,-1,0,0}, dy[] = {0,0,1,-1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if ((i+j)%2 || grid[i][j]!='#') continue;
            for (int d = 0; d < 4; d++) {
                int ni=i+dx[d], nj=j+dy[d];
                if (ni<0||ni>=n||nj<0||nj>=m||grid[ni][nj]!='#') continue;
                g.add_edge(i*m+j, ni*m+nj, 1);
            }
        }
}

```

```

}
}
cout << g.flow(s, t) << "\n";
}

```

Min-cut extraction

```

g.flow(s, t);
auto cut = g.min_cut(s);
// cut[v] == true -> v is on the s-side of the minimum cut
// sum of capacities of cut edges == max flow (max-flow min-cut theorem)

```

15. mincostflow — Min-Cost Max-Flow [Expert] — #include

Send maximum flow from s to t at minimum total cost. Exposes the full cost-flow curve via slope(). Uses shortest-path augmentation with potentials (SPFA).

```

mcf_graph<Cap, Cost> g(int n) // Cap, Cost: int or ll. 0(n)
int g.add_edge(int from, int to, Cap cap, Cost cost) // cap>=0, cost>=0. 0(1)
pair<Cap, Cost> g.flow(int s, int t) // (total_flow, total_cost)
pair<Cap, Cost> g.flow(int s, int t, Cap limit) // stops at limit
vector<pair<Cap, Cost>> g.slope(int s, int t) // piecewise-linear curve
// Complexity: O(F * (n+m) * log(n+m)) where F = total flow

```

AC example (practice2/e) — assignment problem

```

#include <atcoder/mincostflow>
#include <iostream>
using namespace atcoder;
const long long BIG = 1e9;

int main() {
    int n, k; cin >> n >> k;
    // source=2n, sink=2n+1, rows=0..n-1, cols=n..2n-1
    mcf_graph<int, long long> g(2*n+2);
    int s = 2*n, t = 2*n+1;

    g.add_edge(s, t, n*k, BIG); // waste edge to ensure max flow = n*k
    for (int i = 0; i < n; i++) {
        g.add_edge(s, i, k, 0);
        g.add_edge(n+i, t, k, 0);
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            long long a; cin >> a;
            g.add_edge(i, n+j, 1, BIG - a); // negate: max profit -> min cost
        }
}

```

```

auto [flow, cost] = g.flow(s, t, n*k);
cout << 1LL*n*k*BIG - cost << "\n";

for (auto e : g.edges())
if (e.from != s && e.to != t && e.flow == 1)
cout << "row " << e.from << " -> col " << e.to-n << "\n";
}

```

slope() — reading the cost curve

```

auto pts = g.slope(s, t);
// pts[i] = (flow_amount, total_cost_at_that_flow)
// Consecutive difference in cost / difference in flow = marginal cost per unit.
// Use when you want to buy exactly k units at minimum cost, or find the
// optimal trade-off point between flow and cost.
for (auto [f, c] : pts)
printf("flow=%lld cost=%lld\n", (long long)f, (long long)c);

```

Warning: `slope()` and `flow()` can be called only once. All caps and costs must be non-negative.

Quick Reference

Header	Purpose	Difficulty	Complexity
<atcoder/modint>	Modular arithmetic, pow, inv	Beginner	$O(\log \text{mod})$ inv/div
<atcoder/math>	pow_mod, inv_mod, CRT, floor_sum	Easy	$O(\log n)$ / $O(\log m)$
<atcoder/convolution>	Polynomial multiplication (NTT)	Medium	$O(n \log n)$
<atcoder/fenwicktree>	Point-add, prefix-sum	Beginner	$O(\log n)$
<atcoder/segtree>	Point-set, range monoid product	Medium	$O(\log n)$
<atcoder/lazysegtree>	Range-apply, range product	Hard	$O(\log n)$
<atcoder/string>	Suffix array, LCP, Z-algorithm	Med / Hard	$O(n)$ or $O(n \log n)$
<atcoder/dsu>	Union-Find, connectivity	Beginner	$O(\alpha(n))$
<atcoder/scc>	SCCs in topological order	Medium	$O(V + E)$
<atcoder/twosat>	2-SAT satisfiability	Hard	$O(N + M)$
<atcoder/maxflow>	Max flow / min cut (Dinic)	Hard	$O(n^2 m)$
<atcoder/mincostflow>	Min-cost max-flow	Expert	$O(F(n+m)\log(n+m))$

AtCoder Library (ACL) — CC0 licensed. Official docs: atcoder.github.io/ac-library/production/document_en/